



NYC Taxi ELT Analytics Engineering Project

Overview

This project demonstrates a modern ELT (Extract, Load, Transform) analytics engineering workflow using:

- dbt
- DuckDB
- SQL
- Streamlit
- Plotly
- Parquet

The project processes NYC Yellow Taxi trip data and transforms raw records into analytics-ready business models and interactive dashboards.



Architecture

```
graph TD; A[Raw Parquet Files] --> B[DuckDB Warehouse]; B --> C[dbt Staging Models]; C --> D[dbt Intermediate Models]; D --> E[dbt Mart Models]; E --> F[Streamlit Dashboard];
```



What is ELT?

ETL

Extract → Transform → Load

Data is transformed before loading into storage.

ELT

Extract → Load → Transform

Raw data is loaded first into the warehouse, and transformations happen inside the warehouse using SQL.

This is the modern analytics engineering approach used in:

- BigQuery
 - Snowflake
 - Redshift
 - Databricks SQL
 - dbt workflows
-



What is dbt?

dbt = Data Build Tool

A framework used for:

- SQL transformations
 - Data modeling
 - Data quality testing
 - Documentation generation
 - Dependency management
 - Lineage tracking
-



What is DuckDB?

DuckDB is a lightweight analytical database optimized for:

- OLAP analytics
- SQL queries
- Parquet processing
- Local warehouse simulation

DuckDB can directly query:

- Parquet
- CSV
- JSON

without requiring a traditional database server.

Project Structure

```
ELT_nyc_taxi_dbt/
|
├── analyses/
├── data/raw/
├── docs/
├── macros/
├── models/
│   ├── staging/
│   ├── intermediate/
│   └── marts/
├── sources.yml
├── tests/
├── dashboard.py
├── dbt_project.yml
├── profiles.yml
├── README.md
└── .gitignore
```

Layered Modeling

Layer	Purpose	Description
staging	Raw cleaning	Cleans and validates raw data
intermediate	Business logic	Creates features and derived metrics
marts	Reporting layer	Analytics-ready KPI tables
tests	Data quality	Validates data correctness
macros	Reusable logic	Shared SQL business logic
analyses	Ad hoc analysis	Exploratory analytical SQL
docs	Documentation	Metadata and lineage
lineage	Dependency graph	Shows model relationships

Staging Layer

Folder:

```
models/staging/
```

Models:

- stg_taxi_trips
- stg_taxi_zones

Responsibilities:

- remove invalid fares
 - remove impossible distances
 - validate payment types
 - clean timestamps
-

2 Intermediate Layer

Folder:

```
models/intermediate/
```

Models:

- int_trip_features
- int_trip_quality

Generated Features:

- trip_duration_min
 - pickup_hour
 - avg_speed_mph
 - tip_pct
 - is_airport_trip
 - is_weekend
 - time_of_day
-

3 Mart Layer

Folder:

```
models/marts/
```

Models:

- fact_trips
- daily_metrics
- airport_trip_analysis
- payment_type_analysis

Responsibilities:

- KPI reporting
- analytics-ready datasets
- business aggregations

Data Quality Testing

Implemented dbt tests:

- not_null
- accepted_values
- custom SQL assertions

Examples:

- valid payment types
- positive trip duration
- non-null fares

dbt Lineage

Models reference each other using:

```
{{ ref('model_name') }}
```

This creates:

- dependency DAGs
- execution ordering
- lineage graphs



Dashboard Features

The Streamlit dashboard includes:

- KPI cards
 - Business metrics
 - Payment analysis
 - Airport trip analytics
 - Time-of-day trends
 - Heatmaps
 - SQL query playground
 - CSV export
 - Interactive filters
-



Commands Used

Create Virtual Environment

```
python3.11 -m venv venv
```

Activate Environment

```
source venv/bin/activate
```

Install Dependencies

```
pip install dbt-duckdb streamlit duckdb pandas plotly
```

Verify dbt Setup

```
dbt debug
```

Run dbt Models

```
dbt run
```

Run dbt Tests

```
dbt test
```

Generate Documentation

```
dbt docs generate
```

Serve Documentation

```
dbt docs serve
```

Open:

```
http://localhost:8080
```

Run Streamlit Dashboard

```
streamlit run dashboard.py
```



Output Models

Model	Type
stg_taxi_trips	View
stg_taxi_zones	View
int_trip_features	View
int_trip_quality	View
fact_trips	Table
daily_metrics	Table
airport_trip_analysis	Table
payment_type_analysis	Table

Analytics Engineering Concepts Demonstrated

- ELT Architecture
 - Warehouse-Native SQL
 - Layered Data Modeling
 - dbt Transformations
 - Data Quality Testing
 - Analytics Marts
 - Lineage DAG Modeling
 - SQL Dependency Management
 - Interactive Dashboards
-

Interview Talking Points

This project demonstrates:

- ETL → ELT migration thinking
 - Modern analytics engineering workflows
 - Warehouse-native transformations
 - SQL business modeling
 - Data quality engineering
 - Dashboard integration
 - dbt lineage management
-

Final Summary

This project successfully adapts a Spark-based ETL pipeline into a modern analytics engineering ELT workflow using dbt and DuckDB.

The pipeline transforms raw NYC taxi parquet data into analytics-ready business models using modular SQL transformations, automated testing, lineage-driven dependency management, and interactive dashboarding.